

Agent Core 原生工具集智能体架构设计规范

解耦硬编码关键词 · Native Tool Calling · ReAct 循环 · 事件驱动与沙盒隔离

Architecture Spec v2.0

ReAct State Machine

OpenAI Function Calling

Electron + Python Sandbox

核心设计思想转型 (Paradigm Shift)

从传统的“基于关键词正则匹配与强提示词 JSON 解析的脚本调度器”升级为“原生 Function Calling 驱动的自主 Agent”。智能体通过 ReAct (Reasoning & Acting) 状态机循环独立完成目标拆解、工具选择、沙盒执行、结果观察与自我纠错。

一、Agent Core 系统整体架构图

分层结构	核心模块组件	职责说明
交互展示层 (Electron / Web)	- Frontend UI (React/Vue) - WebSocket Client - Stream Terminal & Project Tree	负责用户指令输入、流式思考 (Reasoning) 与正文展示、工具调用状态推送与中断控制。
通讯与总线层 (Event Gateway)	- EventBus (Pub/Sub) - WebSocket Manager - Interrupt Token Manager	统一管理异步事件推送，提供 `Thought`, `Action`, `Observation`, `Error`, `Artifact` 等标准化事件。
Agent 核心内核 (Agent Core)	- AgentLoop (ReAct 循环) - ToolRegistry (工具注册表) - ContextManager (上下文与 Token)	- 维护自主 Task 状态机循环 - 将 SKILL.md 动态编译为 Native Tool Schemas - 管理 Token 预算与滑动历史窗口
执行与适配层 (Execution & Adapters)	- SandboxExecutor (沙盒环境) - LLMClient (模型统一适配器) - Subprocess Wrapper	- 独立隔离子进程执行 Tool，捕获 stdout/stderr/错误栈 - 统一支持 OpenAI / Claude / MiniMax 的 Function Calling
底层资源库 (Resources)	`.workbuddy/skills/*` `.workspace/projects/*`	本地技能库目录与音乐工程文件目录。

二、Core 模块完整文件夹与文件目录树

在后端架构中，将 `app/core/` 彻底重构为职责清晰的模块化设计：

```
app/core/
├── __init__.py           # 暴露 Core 统一导出接口 (AgentCore, AgentLoop, EventBus)
├── agent_loop.py         # 核心: ReAct (Reasoning & Acting) 状态机循环与生命周期管理
├── tool_registry.py      # 工具中心: 解析 SKILL.md, 动态生成 OpenAI Function Calling Schema
├── sandbox_executor.py   # 沙盒执行器: 隔离执行 Python/Subprocess, 捕捉 stdout/stderr 与异常堆栈
├── event_bus.py         # 事件总线: 发布/订阅模式, 处理实时流式事件推送与前端 WebSocket 对接
├── context_manager.py    # 上下文管理: Token 预算裁剪、系统提示词注入、动态工程状态感知
├── llm_client.py        # 适配层: 统一支持 Native Function Calling & Reasoning/Thinking 标签提取
├── interrupt_token.py    # 控制中断: 用户点击“暂停/取消”时的异步 Task 中断与沙盒进程 Kill
├── schemas/             # 结构化 Pydantic 数据类型定义
│   ├── __init__.py
│   ├── events.py        # 各种 Event 定义 (ThoughtEvent, ActionEvent, ObservationEvent...)
│   └── tools.py         # Tool, ParameterSpec, FunctionCallSpec 定义
```

```
|   └─ agent_state.py          # TaskState, StepResult, LoopStatus 枚举
└─ utils/                     # 核心辅助工具集
    └─ __init__.py
    └─ frontmatter_parser.py  # 高鲁棒性 SKILL.md YAML Frontmatter 解析器
    └─ process_helpers.py    # UTF-8/Windows 编码绕过包装器与跨平台子进程拉起
    └─ prompt_templates.py   # 基础 Agent System Prompt 与 ReAct 指引模板
```

三、核心交互与 ReAct 循环流

Step 1: 上下文感知与工具注册 (Context & Tools Load)

ToolRegistry 自动扫描 `.workbuddy/skills/`，将各个技能的 SKILL.md 定义编译为标准 JSON Schema。ContextManager 动态注入当前工程状态。

Step 2: 发起模型调用 (Model Call with Native Tools)

LLMClient 带着 `messages` 和 `tools` 参数发起流式请求。流式接收 `reasoning\_content` (思考过程) 并通过 EventBus 实时推送前端。

Step 3: 判定工具调用 (Action Decision)

模型输出 `tool\_calls`。AgentLoop 暂停文字生成，触发 `ActionEvent`。如果无需工具调用，输出正文并结束循环。

Step 4: 沙盒隔离执行 (Sandbox Execution)

SandboxExecutor 根据 tool_name 分发：对于可执行脚本调起子进程隔离运行，捕获日志与产物；对于纯提示词技能交由模型子句执行。

Step 5: 反馈观察与自主自我纠错 (Observation & Self-Correction)

将执行日志与产物（或 Traceback 报错信息）包装为 `role: "tool"` 消息追加回上下文，再次进入 Step 2。如报错，LLM 会自主分析错误并修正参数重新重试。

四、关键组件接口规范摘要

1. ToolRegistry (工具注册中心)

```
class ToolRegistry:
    def scan_skills(self) -> List[ToolSpec]: ...
    def get_openai_tools_schema(self) -> List[dict]: ...
    def get_skill(self, tool_name: str) -> Optional[SkillMeta]: ...
```

2. AgentLoop (ReAct 循环控制器)

```
class AgentLoop:
    async def run_task(
        self,
        user_prompt: str,
        project_name: str,
        interrupt_token: InterruptToken
    ) -> AsyncGenerator[AgentEvent, None]: ...
```

3. EventBus (事件定义)

```
class EventType(Enum):  
    THOUGHT = "thought"          # 思考过程流  
    ACTION = "action"           # 决定调用工具  
    LOG = "log"                  # 工具运行实时日志  
    OBSERVATION = "observation" # 工具运行结果/观察  
    TEXT = "text"                # 最终回复正文  
    ERROR = "error"              # 异常事件  
    ARTIFACT = "artifact"        # 生成工程产物通知
```